

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology



Natural Language Processing

Vietnamese Legal RAG

Submitted by:

Name: Nguyen Hoang Nam
ID: 202416727
Email: nam.nh2416727@sis.hust.edu.vn

Name: Nguyen Ngoc Tuan Anh
ID: 202416658
Email: anh.nnt2416658@sis.hust.edu.vn

Name: Dang Xuan Bach
ID: 202416664
Email: bach.dx2416664@sis.hust.edu.vn

Name: Truong Tue Minh
ID: 202416719
Email: minh.tt241719@sis.hust.edu.vn

Name: Pham Xuan Hieu
ID: 202416692
Email: hieu.px2416692@sis.hust.edu.vn

Group: 01
Course ID: 168071
Semester: 20252
Instructor: Assoc. Prof. Le Thanh Huong, Dr. Nguyen Kiem Hieu

Ngày 1 tháng 6 năm 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Contributions	4
2	Dataset	4
2.1	Dataset Overview	4
2.2	Dataset Statistics	5
2.3	Legal Sector Filtering	5
2.4	Data Preprocessing	5
3	Background	5
3.1	Retrieval-Augmented Generation	5
3.2	Dense Retrieval with BGE-M3	6
3.3	BM25 Lexical Retrieval	6
3.4	Hybrid Retrieval via Reciprocal Rank Fusion	6
3.5	Legal Citation Graph	6
3.6	Rule-Based Reranking	7
3.7	Answer Generation with Qwen 2.5-Instruct	7
4	Data Preprocessing Pipeline	7
4.1	Dataset Acquisition and Filtering	7
4.2	Document Cleaning and Structured Chunk Extraction	7
4.2.1	Text Normalisation	7
4.2.2	Hierarchical Legal Structure Parsing	8
4.2.3	Post-Parsing Filters and Metadata	8
4.3	Quality Inspection	8
5	Dense Passage Embedding and Vector Store Construction	8
5.1	Embedding Model	8
5.2	Tokenisation and CLS Pooling	9
5.3	L2 Normalisation	9
5.4	Multi-GPU Parallelism	9
5.5	FAISS Flat Inner-Product Index	9
5.6	Persisted Artefacts	9
6	Legal Knowledge Graph Construction	9
6.1	Configuration and Data Loading	10
6.2	Text Normalisation Utilities	10
6.3	Document Node Construction	10
6.4	Citation Mention Extraction	10
6.5	Citation Resolution	11
6.6	Edge Deduplication and Graph Serialisation	11
6.7	LLM-Based Edge Validation	11
7	Retrieval Pipeline Architecture	11
7.1	BM25 Lexical Retrieval	11
7.2	Dense Vector Retrieval	12
7.3	Hybrid Reciprocal Rank Fusion	12
7.4	Legal Citation Graph Expansion	12
7.5	Rule-Based Reranking	12
8	Answer Generation	12
8.1	Model Selection and Quantisation	12
8.2	Context Formatting and Prompt Construction	12
8.3	Inference	13

9	Evaluation Framework	13
9.1	Synthetic Benchmark Construction	13
9.2	Retrieval Evaluation Metrics	13
10	System Implementation	14
10.1	Retrieval Module	14
10.2	Retrieval-Augmented Generation Pipeline	15
10.3	Offline Evaluation Harness	15
10.4	Sample Output of the RAG Pipeline	16
11	Results and Analysis	17
11.1	Quantitative Comparison	18
11.2	Why Does Graph Expansion Hurt Performance?	18
11.3	Why Reranking Helps Only Marginally	18
11.4	Recall@100 and Recall@200	19
12	Further Improvements	19
12.1	Learnable Graph Expansion with Pruning	19
12.2	Cross-Encoder Re-Ranking as a Second Stage	19
12.3	Chunk-Level Graph Instead of Document-Level	19
12.4	Domain-Adaptive Fine-Tuning of the Dense Retriever	19
12.5	Better Handling of Multi-Hop Queries	20
12.6	Evaluation on Real User Queries	20
12.7	Real-Time Deployment Optimisations	20

Abstract

Vietnamese legal documents are characterised by complex hierarchical structures, specialised terminology, and extensive cross-references between regulations, which pose significant challenges for information retrieval and question answering systems. In this project, we develop a Retrieval-Augmented Generation (RAG) system tailored to the Vietnamese legal domain [5]. The proposed system employs **article-level chunking** to preserve the semantic and structural integrity of legal provisions. To improve retrieval performance, we adopt a hybrid retrieval strategy that combines **semantic search** [4] using the BGE-M3 multilingual bi-encoder [1] and **BM25 lexical retrieval** [3], fused via Reciprocal Rank Fusion [13]. In addition, a **citation graph** is constructed to model relationships among legal documents and provisions, enabling retrieval of relevant information through legal references [14]. Finally, a deterministic rule-based reranker encodes domain knowledge to refine the retrieved results and select the most relevant passages for answer generation by Qwen 2.5-Instruct [17]. The integration of these techniques aims to improve the accuracy, relevance, and reliability of legal question answering for Vietnamese legal texts.

1 Introduction

1.1 Motivation

Vietnamese legal documents are highly structured and contain complex hierarchical relationships, including laws, decrees, circulars, and detailed articles and clauses [19]. These documents are frequently updated and heavily interconnected through citations and references. As a result, retrieving accurate legal information using traditional keyword-based search methods is challenging, especially for non-expert users.

With the rapid growth of digital legal databases in Vietnam, there is an increasing demand for intelligent systems that can support legal question answering with high accuracy and contextual understanding. This motivates the development of Retrieval-Augmented Generation (RAG) systems that combine information retrieval and natural language generation to provide reliable answers grounded in legal texts [6].

1.2 Problem Statement

Despite advances in neural retrieval models, Vietnamese legal question answering still faces several challenges. First, legal texts are long and structured, making naive chunking methods ineffective at preserving semantic boundaries. Second, purely dense retrieval methods [4] may fail to capture exact legal references such as article numbers or legal codes, while lexical methods like BM25 [3] lack semantic understanding. Third, relationships between legal documents are often implicit and scattered across multiple sources.

Therefore, there is a need for a unified retrieval framework that can effectively combine structural awareness, semantic understanding, lexical matching, and inter-document relationships.

1.3 Objectives

This project aims to design and implement a Vietnamese Legal Retrieval-Augmented Generation system with the following objectives:

- Develop an **article-level chunking strategy** to preserve the structure of legal documents.
- Implement a **hybrid retrieval system** combining semantic search (Bi-Encoder [1]) and BM25-based lexical search [2].
- Construct a **citation graph** to model relationships between legal documents and provisions [15].
- Apply a **rule-based reranker** to improve retrieval precision using domain knowledge.
- Integrate all components into a unified RAG pipeline for legal question answering [5].

1.4 Contributions

The main contributions of this project include:

- A structured pipeline for Vietnamese legal document processing and retrieval [19].
- A hybrid retrieval mechanism that balances semantic and lexical matching via Reciprocal Rank Fusion [13].
- A graph-based expansion method leveraging citation relationships in legal texts [16].
- An end-to-end RAG system optimised for Vietnamese legal question answering, powered by Qwen 2.5-Instruct [17].

2 Dataset

2.1 Dataset Overview

In this project, we utilise the **Vietnamese Legal Documents Dataset** published on Hugging Face [22]:

<https://huggingface.co/datasets/minhdoan17/vietnamese-legal-documents>

The dataset is a large-scale Vietnamese legal corpus consisting of more than one million legal document entries, covering various types of legal texts such as laws, decrees, circulars, and official decisions. Each document contains full textual content along with metadata such as document ID, issuing authority, and legal sector classification.

The dataset is widely used for tasks such as information retrieval, question answering, and legal text mining. It provides a realistic and diverse collection of Vietnamese legal language, making it suitable for building Retrieval-Augmented Generation systems.

2.2 Dataset Statistics

The full dataset contains over 1,000,000 legal document entries, with each entry typically consisting of long-form legal text. The distribution of documents spans multiple domains of law, including administrative law, civil law, criminal law, and economic regulations.

Formally, let the dataset be defined as:

$$\mathcal{D} = \{(x_i, s_i)\}_{i=1}^N \quad (1)$$

where x_i is the raw legal document text, s_i is the associated metadata (including legal sector), and $N \approx 10^6$ is the total number of documents.

2.3 Legal Sector Filtering

Due to computational constraints and limited training resources, we restrict the dataset to a subset of documents belonging to a specific **legal sector**. Let $\mathcal{S}$ denote the set of all legal sectors and let $s^* \in \mathcal{S}$ be the selected target sector. We construct a filtered dataset:

$$\mathcal{D}_{s^*} = \{(x_i, s_i) \in \mathcal{D} \mid s_i = s^*\} \quad (2)$$

This filtering strategy reduces computational cost for embedding and retrieval indexing [7], allows faster experimentation with Bi-Encoder and Cross-Encoder models [10], and improves evaluation consistency by focusing on a single legal domain.

2.4 Data Preprocessing

Before building the retrieval system, the raw legal documents are processed through several steps:

1. **Text Cleaning:** Removal of unnecessary whitespace, special characters, and formatting artefacts.
2. **Normalisation:** Standardising Vietnamese text encoding and legal abbreviations [19].
3. **Chunking:** Splitting documents into smaller units based on legal structure, primarily at the **article (Dieu)** level.

Let a document x_i be split into chunks:

$$x_i \rightarrow \{c_{i1}, c_{i2}, \dots, c_{im}\} \quad (3)$$

where each chunk c_{ij} corresponds to a semantically coherent legal unit. This article-level chunking preserves the hierarchical structure of legal texts, ensuring that each chunk contains complete legal meaning rather than arbitrary splits.

3 Background

3.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [5] is a framework that combines information retrieval with large language models to improve factual correctness and reduce hallucination. Given a query q , the system retrieves a set of relevant document chunks $D = \{d_1, d_2, \dots, d_k\}$ from a large corpus $\mathcal{C}$ and uses them as external knowledge to condition the answer generator:

$$a = \arg \max_a P(a \mid q, D) \quad (4)$$

The key idea is that the parametric knowledge of the model is complemented by non-parametric external memory [6]. In our system the generator is Qwen 2.5-Instruct [17] and the retrieval corpus is the filtered Vietnamese land-law chunk store.

3.2 Dense Retrieval with BGE-M3

Dense retrieval maps queries and documents into a shared vector space using a Bi-Encoder [4]. We use BAAI/bge-m3 [1], a multilingual model based on the XLM-RoBERTa architecture [9] that was pre-trained on a corpus covering Vietnamese. Each text t is encoded to a CLS-pooled, ℓ_2 -normalised vector:

$$\mathbf{e} = \frac{\text{CLS}(\text{BGE-M3}(t))}{\|\text{CLS}(\text{BGE-M3}(t))\|_2} \in \mathbb{S}^{d-1}, \quad d = 1,024 \quad (5)$$

Similarity is computed as an inner product, which equals cosine similarity for unit vectors. Retrieval over the full corpus is accelerated by a FAISS IndexFlatIP [7], which performs exact inner-product search with complexity $\mathcal{O}(N \cdot d)$ per query.

3.3 BM25 Lexical Retrieval

BM25 [3] is a probabilistic retrieval model based on term frequency and inverse document frequency. For a query $Q = \{q_1, \dots, q_n\}$ and a document d , the score is:

$$\text{BM25}(d, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} \quad (6)$$

with $k_1 = 1.5$ and $b = 0.75$. BM25 is particularly effective in legal retrieval because queries often contain exact identifiers such as article numbers or decree codes (e.g. “*Điều 12*”, “*Nghị định 15/2020*”) that dense-only models may miss. Vietnamese word segmentation is handled by PyVi ViTokenizer [20], and both the inverted index and the tokenised corpus are cached to disk to avoid re-processing across sessions.

3.4 Hybrid Retrieval via Reciprocal Rank Fusion

Semantic and lexical retrieval complement each other [11, 12]. We fuse the two ranked lists using Reciprocal Rank Fusion (RRF) [13]. Given the BM25 rank $r_{\text{bm25}}(c)$ and vector rank $r_{\text{vec}}(c)$ of a candidate chunk c , the fused score is:

$$s_{\text{fused}}(c) = \frac{1 - \alpha}{k + r_{\text{bm25}}(c)} + \frac{\alpha}{k + r_{\text{vec}}(c)} \quad (7)$$

where $k = 60$ is the standard RRF offset and $\alpha \in [0, 1]$ controls the relative weight of the vector component ($\alpha = 0.6$ by default). This hybrid mechanism improves robustness across different query types, which is especially important in legal domains where both semantic paraphrasing and exact wording carry relevance signals.

3.5 Legal Citation Graph

Legal documents naturally form a citation network. We model this as a directed graph [16, 14]:

$$G = (V, E) \quad (8)$$

where V is the set of legal documents and each directed edge $(v_i, v_j) \in E$ indicates that v_i references v_j with an associated relation type (e.g. *replaces*, *amends*, *guides*, *cites*) and a confidence score produced by our citation-resolution pipeline. Given the initial fused result set, graph expansion retrieves additionally relevant documents:

$$V_{\text{expanded}} = V_0 \cup \{v_j \mid (v_i, v_j) \in E, v_i \in V_0\} \quad (9)$$

Graph expansion is particularly useful for legal QA because many complete answers require combining multiple interconnected provisions [15].

3.6 Rule-Based Reranking

After graph expansion, a deterministic reranker applies multiplicative factors based on domain knowledge: a legal-type prior that promotes statutes over administrative circulars, a locality filter that boosts or penalises provincial documents depending on whether the query mentions a matching locality, an intent-based penalty for chunks that diverge from the inferred query purpose, and a direct-answer boost for chunks containing high-density positive terms. This layer encodes expert heuristics that are difficult to learn from limited training data, complementing the statistical retrieval signals produced by the BM25–vector fusion stage.

3.7 Answer Generation with Qwen 2.5-Instruct

Retrieved and reranked chunks are assembled into a context window (bounded to 9,000 characters) and passed to Qwen 2.5-Instruct [17] via a two-message chat template. The system prompt instructs the model to act as a Vietnamese land-law assistant, answer solely from the provided sources, prefer more recent or higher-level legal instruments when amendment or replacement relations exist, and cite sources by chunk ID. Inference uses the Hugging Face `transformers` library [18] with optional AWQ [23] or GPTQ [24] quantisation for deployment on memory-constrained hardware.

4 Data Preprocessing Pipeline

This section describes the three-stage preprocessing pipeline developed to transform raw Vietnamese legal documents into structured, semantically meaningful chunks suitable for retrieval-augmented generation.

4.1 Dataset Acquisition and Filtering

Raw data is sourced from the `minhdoan17/vietnamese-legal-documents` [22] dataset on Hugging Face, which consists of two separate configurations: `metadata` and `content`. Both are loaded in streaming mode to avoid excessive memory consumption.

Metadata Filtering. The metadata stream is scanned to retain only documents relevant to land and real-estate law. Concretely, a document is kept if and only if:

1. its `legal_sectors` field contains the token “**bat dong san**” (real estate),
2. its `title` field contains the token “**dat**” (land), and
3. its `issuance_date` is on or after **1 January 2010**.

The date threshold intentionally excludes superseded or outdated regulations, keeping the corpus aligned with the current Vietnamese land-law framework.

Content Joining. After filtering, the content stream is scanned once and every record whose `id` matches a retained metadata entry is merged into a single dictionary using a metadata-first, content-override strategy. The merged records are serialised to a Parquet file (`LandLawDocument.parquet`) via `pandas` [21], which serves as the input to the subsequent cleaning stage.

4.2 Document Cleaning and Structured Chunk Extraction

The cleaning stage reads `LandLawDocument.parquet` with Polars [21] and applies a deterministic pipeline to each document before splitting it into fine-grained chunks.

4.2.1 Text Normalisation

Three normalisation passes are applied sequentially:

1. **Unicode normalisation** (NFC): resolves composed/decomposed diacritic variants common in Vietnamese text [19] and strips zero-width spaces, non-breaking spaces, and the byte-order mark.
2. **Boilerplate removal**: recurring header and footer strings—such as the national motto, page-number tokens, and signatory titles—are matched with regular expressions and replaced with a single space.

3. **Whitespace normalisation:** carriage returns are converted to newlines, runs of three or more consecutive newlines are collapsed to two, and runs of multiple horizontal spaces are reduced to a single space.

4.2.2 Hierarchical Legal Structure Parsing

Vietnamese legislation follows a strict three-level hierarchy: *Dieu* (Article) $\supset$ *Khoan* (Clause) $\supset$ *Diem* (Point). Three cascading regex parsers exploit this structure to extract article boundaries, numbered clauses, and lettered points. A key heuristic prevents over-fragmentation: if any individual point contains fewer than 50 characters, all points belonging to that clause are grouped into a single chunk at the clause level.

4.2.3 Post-Parsing Filters and Metadata

Two content-based filters are applied before chunks are retained. Administrative tail chunks (signature blocks, appendices, recipient lists) are discarded, while table-like chunks are preserved with a `chunk_type = "table"` label. Cross-references of the form “*Dieu N [law name] [year]*” are extracted and stored as structured triples in a `references` field for downstream graph construction. Each chunk is assigned a `chunk_id` of the form `{document_id}_{index}` and an MD5 content hash; global deduplication is then performed using Polars [21] `unique` on the hash column.

4.3 Quality Inspection

A lightweight inspection script reads the cleaned parquet, computes per-chunk character counts, and draws a random sample of 30 chunks whose content exceeds 500 characters for manual verification. Table 1 provides an overview of the corpus at each stage.

Bảng 1: Corpus statistics at each preprocessing stage.

Stage	Count	Unit
Raw documents after metadata filter	—	documents
Merged documents (<code>LandLawDocument.parquet</code>)	—	documents
Chunks before deduplication	—	chunks
Chunks after deduplication (<code>LandLawDocumentCleaned.parquet</code>)	—	chunks
Chunks with ≥ 500 characters	—	chunks

The exact counts should be filled in after running the full pipeline on the complete dataset.

5 Dense Passage Embedding and Vector Store Construction

This section describes how the cleaned text chunks produced in Section 4 are encoded into dense vector representations and indexed for sub-linear approximate nearest-neighbour retrieval. The pipeline uses the **BGE-M3** bi-encoder model [1] and a flat inner-product FAISS index [7], with optional multi-GPU parallelism.

5.1 Embedding Model

The encoder is `BAAI/bge-m3` [1], a multilingual dense retrieval model based on the XLM-RoBERTa architecture [9]. BGE-M3 was chosen for three reasons specific to the Vietnamese legal domain: (1) it was pre-trained and fine-tuned on a multilingual corpus covering Vietnamese; (2) it supports a maximum sequence length of 8,192 tokens (here truncated to 512 for throughput); and (3) its training objective combines dense, multi-vector, and sparse retrieval losses, yielding a representation that generalises well across heterogeneous legal text types.

The model is loaded in half-precision (`torch.float16`) with the scaled dot-product attention (SDPA) backend:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (10)$$

where d_k is the per-head dimension.

5.2 Tokenisation and CLS Pooling

Each text chunk t_i is tokenised with the BGE-M3 WordPiece tokeniser, truncated or padded to $L = 512$ tokens. The encoder maps the input to contextualised token representations:

$$\mathbf{H} = \text{Encoder}(\mathbf{X}, \mathbf{M}) \in \mathbb{R}^{B \times L \times d} \quad (11)$$

where $d = 1,024$. The sentence-level embedding is obtained by CLS pooling:

$$\tilde{\mathbf{e}}_i = \mathbf{H}_{i,0,:} \in \mathbb{R}^d \quad (12)$$

5.3 L2 Normalisation

Before storage, each raw embedding is ℓ_2 -normalised:

$$\mathbf{e}_i = \frac{\tilde{\mathbf{e}}_i}{\|\tilde{\mathbf{e}}_i\|_2} \in \mathbb{S}^{d-1} \quad (13)$$

For unit vectors, inner product equals cosine similarity, allowing the FAISS index to use the cheaper inner-product operation while preserving cosine semantics.

5.4 Multi-GPU Parallelism

When $G \geq 2$ GPUs are available, the corpus is partitioned into G equal subsets processed in parallel threads. Partial embedding matrices are concatenated along the sample axis:

$$\mathbf{E} = [\mathbf{E}_0^\top \quad \dots \quad \mathbf{E}_{G-1}^\top]^\top \in \mathbb{R}^{N \times d} \quad (14)$$

providing near-linear wall-clock speedup on homogeneous hardware.

5.5 FAISS Flat Inner-Product Index

The embedding matrix is indexed with FAISS `IndexFlatIP` [7], a brute-force exact index. Retrieval solves:

$$\hat{\mathcal{R}}_k(\mathbf{q}) = \underset{i \in [N]}{\text{top-}k} \mathbf{q}^\top \mathbf{e}_i = \underset{i \in [N]}{\text{top-}k} \cos \theta_{qi} \quad (15)$$

A flat index is preferred over approximate alternatives (IVF, HNSW) because exact retrieval is critical for high-stakes legal queries, the corpus fits comfortably in server-class RAM (≈ 4 GB for 10^6 vectors at $d = 1,024$, `float32`), and no offline training phase is required.

5.6 Persisted Artefacts

Three files are written to `data/vector_store/`:

- `faiss_index.bin` — serialised `IndexFlatIP` object.
- `metadata.pkl` — ordered list of `chunk_id` strings and full metadata records.
- `config.json` — human-readable embedding configuration.

Table 2 summarises the key parameters.

6 Legal Knowledge Graph Construction

This section describes the pipeline for constructing a legal knowledge graph from the cleaned chunk corpus produced in Section 4. The pipeline proceeds through five stages: document node construction, citation mention extraction, citation resolution into directed edges, edge deduplication and filtering, and LLM-based edge validation. The approach is motivated by graph-RAG literature [14, 16, 15] and adapted to the specifics of Vietnamese legislative language.

Bảng 2: Embedding and indexing configuration.

Parameter	Value
Encoder model	BAAI/bge-m3 [1]
Hidden dimension d	1,024
Max sequence length L	512 tokens
Pooling strategy	CLS token
Normalisation	ℓ_2 (unit sphere)
Similarity metric	Cosine (via inner product)
Precision	<code>float16</code> (inference), <code>float32</code> (stored)
Batch size per GPU	512 chunks
FAISS index type	<code>IndexFlatIP</code> (exact) [7]

6.1 Configuration and Data Loading

The pipeline is governed by a small set of hyperparameters. The first 30 chunks of each document are always scanned for citation mentions; additional chunks whose content matches at least one of 20 legal citation keywords are also included when `SCAN_KEYWORD_CHUNKS = True`. Only edges with resolution confidence ≥ 0.75 are retained in the final graph; edges resolved through the stricter duplicate-local-number path require a higher threshold of 0.85. A small stratified sample of edges is drawn per relation type and validated by the local LLM.

6.2 Text Normalisation Utilities

A suite of deterministic normalisation functions underpins all downstream matching. Vietnamese diacritics are stripped via NFD decomposition followed by category-Mn character removal [19]. Document numbers are canonicalised (e.g. “*Nghị định số 43/2014/ND-CP*” $\rightarrow$ `43/2014/nd-cp`), and legal type strings are mapped to a closed vocabulary of eight canonical identifiers. Fuzzy string similarity is computed with `difflib.SequenceMatcher`, returning a ratio in $[0, 1]$.

6.3 Document Node Construction

Legal document nodes are derived by grouping the chunk table on `doc_id` and aggregating metadata fields. Each node is enriched with normalised fields, year signals, a province hint extracted from a curated list of 63 Vietnamese province names, and scope flags distinguishing local-government documents (e.g. `qd-UBND`) from central-government documents (e.g. `nd-cp`). A secondary lookup index maps each unique normalised document number to the list of nodes sharing it.

6.4 Citation Mention Extraction

Two complementary regular expressions cover the two major citation forms in Vietnamese legislation: document-number citations of the form “*Nghị định số 43/2014/ND-CP*” and law title–year citations of the form “*Luật Đại hội đồng 2013*”. For each mention, a 350-character context window is extracted and matched against four keyword groups to assign a preliminary relation type, as summarised in Table 3.

Bảng 3: Relation type classification rules based on normalised context keywords.

Relation	Keywords (normalised)
<code>replaces</code>	thay thế, bãi bỏ, hết hiệu lực
<code>amends</code>	sửa đổi, bổ sung, sửa đổi bổ sung
<code>guides</code>	hướng dẫn, quy định chi tiết, thi hành
<code>based_on</code>	can cứ
<code>cites</code>	(default – no keyword matched)

6.5 Citation Resolution

Each raw mention is resolved to a target document node through one of two paths. For document-number citations, candidates are looked up in the number index. When multiple candidates share the same number, a multi-signal scoring function accumulates evidence from five signals, as summarised in Table 4.

Bảng 4: Candidate scoring signals for duplicate-number resolution.

Signal	Score	Condition
Exact number match	+1.0	citation number = candidate number
Legal type match	+0.2	citation type = candidate type
Year match	+0.2	citation year = candidate year
Title similarity	$+0.2 \times \text{sim}$	$\text{sim} \geq 0.65$
Province match	+0.35	province hint in evidence text
Evidence–title overlap	+0.1	token overlap ratio ≥ 0.15

The province match signal is particularly important for local-government documents, which routinely reuse the same number across all 63 provinces.

6.6 Edge Deduplication and Graph Serialisation

After resolution, self-loops are removed and the resolved subset is deduplicated at the document level by grouping on (`source_doc_id`, `target_doc_id`, `relation_type`). The pipeline writes six output files to `GRAPH_DIR`: node table, edge table, unresolved mentions, ambiguous mentions, full mention-level table, and a top-cited-documents ranking. Graph summary statistics are written to `graph_stats.json`.

6.7 LLM-Based Edge Validation

A stratified sample of resolved edges is validated by a locally hosted Qwen2.5-3B-Instruct model [17] running via the Hugging Face `transformers` library [18]. Each prompt is written in Vietnamese and instructs the model to select a relation label from the allowed set and return a structured JSON response. Agreement is computed deterministically by comparing the normalised system relation against the normalised model-predicted relation, bypassing the model’s self-reported confidence to avoid hallucination artefacts.

7 Retrieval Pipeline Architecture

This section describes the implementation of the multi-stage retrieval pipeline. The system is built on five core modules—BM25 lexical retrieval, vector semantic retrieval, hybrid fusion, legal graph expansion, and rule-based reranking—forming a layered architecture that progressively refines candidate chunks extracted from the land-law corpus.

7.1 BM25 Lexical Retrieval

The `BM25Search` module implements the BM25 scoring [3] formula of Eq. (16):

$$\text{BM25}(d, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} \quad (16)$$

with $k_1 = 1.5$ and $b = 0.75$. The inverted index stores per-term posting lists as compressed NumPy arrays, enabling vectorised scoring without iterating over the full corpus. Tokenisation uses PyVi `ViTokenizer` [20] for Vietnamese word segmentation, with whitespace splitting as a fallback. Both the tokenised corpus and the serialised index are cached to disk with file-stat-based validation.

7.2 Dense Vector Retrieval

The **VectorSearch** module encapsulates a FAISS-based nearest-neighbour index [7] over dense embeddings produced by BAAI/bge-m3 [1]. Query embeddings are computed as CLS-pooled, ℓ_2 -normalised vectors (Section 5), and top- k approximate nearest neighbours are retrieved by inner-product search. On GPU the model runs in `float16` precision; CPU inference uses `float32`.

7.3 Hybrid Reciprocal Rank Fusion

The **HybridSearch** module combines BM25 and vector candidate pools via RRF [13]:

$$s_{\text{fused}}(c) = \frac{1 - \alpha}{k + r_{\text{bm25}}(c)} + \frac{\alpha}{k + r_{\text{vec}}(c)} \quad (17)$$

with $k = 60$ and $\alpha = 0.6$ by default. Each result is annotated with raw BM25 and vector scores as well as individual RRF contributions for interpretability.

7.4 Legal Citation Graph Expansion

The **GraphExpansion** module augments the fused candidate set with documents connected to seed results via the citation graph constructed in Section 6. For each seed document d_s with fusion score s_s , the graph score accumulated by a neighbouring document d_t is:

$$g(d_t) = \sum_{e \in \mathcal{E}(d_s, d_t)} \text{conf}(e) \cdot w_{\text{dir}}(\text{rel}(e)) \quad (18)$$

where $w_{\text{dir}}(\cdot)$ is a direction-and-type-specific weight. The final score is:

$$s_{\text{final}}(d) = s_{\text{fused}}(d) \cdot \max(0, 1 + \lambda \cdot \min(g(d), 1) - p(d)) \quad (19)$$

with $\lambda = 0.25$ and a penalty $p(d) \in \{0.10, 0.18\}$ for superseded documents.

7.5 Rule-Based Reranking

The **LegalReranker** module applies multiplicative factors encoding domain knowledge: a legal-type prior (e.g. +0.08 for statutes, -0.08 for administrative letters), a locality filter (boost 1.15 $\times$ for matching province, penalty 0.55 $\times$ for non-matching local documents), intent-based penalties (down to 0.55 $\times$ for off-topic chunks), and direct-answer boosts (up to 1.25 $\times$ for high-density positive terms). All factors are composed multiplicatively so that no single rule can dominate the base retrieval quality.

8 Answer Generation

After retrieval and reranking, the system passes the ranked chunks to the **QwenGenerator** module, which wraps Qwen 2.5-Instruct [17] and synthesises a structured, source-grounded Vietnamese legal answer from the retrieved evidence.

8.1 Model Selection and Quantisation

The generator supports 7B and 14B parameter variants, each available in three quantisation schemes: full-precision, AWQ [23], and GPTQ-Int4 [24]. A two-tier lookup checks a local model cache before falling back to the corresponding Hugging Face Hub identifier, decoupling the pipeline from any hard-coded model path. All generation hyperparameters are collected in the **LLMConfig** dataclass (Table 5).

8.2 Context Formatting and Prompt Construction

Retrieved chunks are formatted into a numbered source block. Sources are appended greedily until the cumulative character count reaches $C_{\text{max}} = 9,000$:

$$\mathcal{C} = \{r_i \in \mathcal{R} \mid \sum_{j=1}^i |\text{fmt}(r_j)| \leq C_{\text{max}}\} \quad (20)$$

The prompt follows a two-message chat template serialised via `apply_chat_template` [18]. The system message instructs the model to answer solely from the provided sources, prefer higher-level and more recent legal instruments, and cite sources by chunk ID.

Bảng 5: Default LLMConfig hyperparameters.

Parameter	Default	Description
model_size	7b	Parameter count variant
quantization	none	Compression scheme (none / awq / gptq)
max_context_chars	9,000	Character budget for retrieved sources
max_new_tokens	700	Maximum generated tokens
temperature	0.2	Sampling temperature
top_p	0.9	Nucleus sampling threshold
device_map	auto	Hugging Face device placement

8.3 Inference

Model loading is performed lazily on the first call to `generate`. Token generation is executed under `torch.inference_mode()`. Sampling is enabled only when `temperature > 0`; otherwise the model generates greedily. Only newly generated tokens are decoded and returned stripped of special tokens and leading/trailing whitespace.

9 Evaluation Framework

To assess retrieval quality, we construct a synthetic benchmark and define standard IR metrics.

9.1 Synthetic Benchmark Construction

Because no large-scale Vietnamese land-law retrieval benchmark exists, we generate a synthetic evaluation set from the corpus using an LLM-in-the-loop procedure.

Chunk sampling. Source chunks are selected from `LandLawDocumentCleaned.parquet` according to a quality filter restricting content length to $[120, 1200]$ characters and excluding administrative noise and low-quality passages. From the filtered pool, $N = 300$ chunks are drawn with a fixed random seed (`RANDOM_SEED = 42`).

Query generation. For each sampled chunk, a prompt is sent to `gpt-4o-mini` requesting $K = 3$ Vietnamese search queries of distinct types: *direct*, *scenario*, and *keyword*. The model responds with a structured JSON object. LLM calls are retried up to three times with exponential back-off on failure.

Query validation. Generated queries undergo a multi-stage validation pipeline:

1. **Length check:** $10 \leq |q| \leq 250$ characters.
2. **Artefact check:** no meta-references to the source text.
3. **Locality consistency:** local-government source chunks require locality mention in the query.
4. **Non-extractiveness:** Jaccard overlap between query and source must not exceed 0.85.
5. **Semantic anchor:** at least three shared non-stopword terms with the source, or one term plus a legal procedure anchor.

Each accepted query is written to `eval_queries.jsonl` alongside the `gold_chunk_ids` field.

9.2 Retrieval Evaluation Metrics

The evaluation module computes three standard metrics over the benchmark set $\mathcal{Q} = \{(q_i, G_i)\}_{i=1}^N$, where G_i is the set of gold chunk IDs for query q_i .

Recall@ k .

$$\text{Recall@}k(q_i) = \begin{cases} 1 & \text{if } \hat{R}_k(q_i) \cap G_i \neq \emptyset \\ 0 & \text{otherwise} \end{cases} \quad \overline{\text{Recall@}k} = \frac{1}{N} \sum_{i=1}^N \text{Recall@}k(q_i) \quad (21)$$

We report Recall@5 and Recall@10.

Mean Reciprocal Rank@ k .

$$\text{MRR@}k = \frac{1}{N} \sum_{i=1}^N \begin{cases} \frac{1}{r} & \text{if } \exists r \leq k : \hat{r}_r(q_i) \in G_i \\ 0 & \text{otherwise} \end{cases} \quad (22)$$

We report MRR@10 as the primary ranking quality metric.

Implementation. The `evaluate_retrieval` function accepts the benchmark records and a dictionary mapping each `query_id` to a ranked list of result dicts, and returns macro-averaged metric values together with a `missing_results` count. The interface is retriever-agnostic: any system returning a list of dicts with a `chunk_id` field can be evaluated without modification.

10 System Implementation

This section describes the three main execution modules of the LandBot system: a standalone retrieval module, a retrieval-augmented generation pipeline, and an offline evaluation harness. Together, these components cover the full lifecycle of the system—from document retrieval to answer generation and systematic quality measurement. All three modules are built on the shared `hybrid_search` engine described in Section 7.

10.1 Retrieval Module

The retrieval module serves as the core information-access layer. Given a natural-language query in Vietnamese, it returns the top- k most relevant text chunks, ranked by the hybrid score of Eq. (17).

Design

The module exposes a CLI with four configurable parameters (Table 6).

Bảng 6: Configurable parameters of the retrieval module.

Parameter	Default	Description
<code>--query</code>	(domain query)	Natural-language question to answer.
<code>--top-k</code>	5	Number of chunks returned.
<code>--alpha</code>	0.4	Weight of the dense vector component in the RRF fusion formula.
<code>--rrf-k</code>	60	Smoothing constant for the RRF re-ranker [13].

Workflow

At runtime, the module (i) parses CLI arguments, (ii) calls `hybrid_search` with standard-output suppressed to prevent library noise, and (iii) prints each retrieved chunk together with its identifier and normalised text content. Wall-clock loading time is measured around the search call so that end-to-end latency can be monitored independently of output formatting.

```
results = hybrid_search(
    args.query,
    top_k=args.top_k,
    alpha=args.alpha,
```

```

    rrf_k=args.rrf_k,
)

```

Listing 1: Core retrieval call inside the retrieval module.

10.2 Retrieval-Augmented Generation Pipeline

The RAG pipeline extends the retrieval module by forwarding the retrieved context to Qwen 2.5-Instruct [17] and returning a free-text answer. This two-stage design—retrieve then generate—keeps the LLM grounded in authoritative legal text while benefiting from its language-generation capabilities [5].

Configuration

All generation hyperparameters are encapsulated in `LLMConfig` (Table 7).

Bảng 7: Key parameters of the RAG pipeline.

Parameter	Default	Description
<code>model-size</code>	7b	Qwen 2.5-Instruct variant [17].
<code>quantization</code>	none	Compression: <code>awq</code> [23], <code>gptq</code> [24], or full.
<code>max-context-chars</code>	9 000	Character budget for the retrieved context (cf. Eq. (20)).
<code>max-new-tokens</code>	700	Maximum tokens in the generated answer.
<code>temperature</code>	0.2	Sampling temperature; low values favour deterministic output.
<code>top-p</code>	0.9	Nucleus-sampling cumulative probability.

Workflow

The pipeline runs in four sequential steps:

1. **Retrieval.** `hybrid_search` is called with the same parameters as the standalone retrieval module.
2. **Context assembly.** The `content` fields of the top- k chunks are concatenated up to `max-context-chars` characters.
3. **Generation.** `QwenAnswerGenerator.generate` prompts the model with the query and assembled context.
4. **Reporting.** Retrieved sources and the generated answer are printed together with overall loading time.

```

generator = QwenAnswerGenerator(llm_config)
answer    = generator.generate(args.query, results)

```

Listing 2: Generation call in the RAG pipeline.

10.3 Offline Evaluation Harness

The evaluation harness provides a reproducible benchmark for the retrieval component, enabling systematic comparison of configurations (e.g. varying α or `top-k`).

Metrics

Three retrieval metrics are reported (Table 8).

Bảng 8: Retrieval metrics reported by the evaluation harness.

Metric	Definition
Recall@5	Fraction of queries for which at least one gold chunk appears in the top-5 results (Eq. (??)).
Recall@10	Same as Recall@5 but with a cutoff of 10.
MRR@10	Mean Reciprocal Rank over the top-10 results; measures how highly the first relevant chunk is ranked (Eq. (??)).

Workflow

For each query, the harness silences library output and calls `hybrid_search`, accumulating results in a dictionary keyed by `query_id`. Once all queries are processed, the dictionary is passed to `evaluate_retrieval` (Section 9.2).

```
for index, item in enumerate(eval_queries, start=1):
    retrieval_results[item["query_id"]] = hybrid_search(
        item["query"], top_k=args.top_k,
        alpha=args.alpha, rrf_k=args.rrf_k,
    )
metrics = evaluate_retrieval(eval_queries, retrieval_results)
```

Listing 3: Core evaluation loop.

10.4 Sample Output of the RAG Pipeline

To illustrate the end-to-end behaviour of the RAG pipeline, this section presents a representative example run. The query used is: “*What are the conditions for issuing a land-use rights certificate?*” (“*Dieu kien cap giay chung nhan quyen su dung dat la gi?*”), a typical inquiry in Vietnamese land-law practice.

System Performance

The pipeline completed in **26.51 seconds**, which includes model loading, hybrid retrieval, context assembly, and answer generation. In production deployments where the model is kept warm in memory, the latency attributable to retrieval and generation alone is substantially lower.

Retrieved Sources

The retrieval stage returned five chunks. Table 9 summarises their identifiers and the legal instruments they originate from.

Bảng 9: Chunks retrieved for the sample query (top-5, $\alpha = 0.4$, RRF- $k = 60$).

Chunk ID	Source / Topic
186844_91	Procedure for issuing land-use rights certificates via the commune-level one-stop shop; responsibilities of the commune People’s Committee, the District Land Registration Office, the Department of Natural Resources and Environment, and the tax authority.
259382_188	Determination of land-use purpose for compensation calculations upon State land reclamation.
364653_84	Community participatory mapping methodology for identifying land categories in a village.
364653_37	Framework for proposing forestry land allocation plans at the commune level.
183503_80	Conditions for issuing certificates for land within an approved urban or rural construction plan, including financial obligations for residential and non-residential land.

Only chunks 186844_91 and 183503_80 are directly relevant to the query; the remaining three were retrieved due to lexical overlap with land-administration terminology. This illustrates a known limitation of hybrid retrieval on broad legal queries [11]: high-frequency domain terms can cause topically adjacent but semantically distant documents to surface in the top- k results.

Generated Answer

The answer produced by Qwen2.5-7B-Instruct [17], grounded in the retrieved context, identifies ten procedural conditions required for the issuance of a land-use rights certificate under Decision 06/2013/QĐ-UBND of Thai Binh Province. The model correctly attributes the source document and `chunk_id`, demonstrating that the citation mechanism functions as intended. The key conditions are summarised in Table 10.

Bảng 10: Procedural conditions for issuing a land-use rights certificate as generated by the RAG pipeline (source: Decision 06/2013/QĐ-UBND, Thai Binh Province).

Step	Condition / Responsible Authority
1	Applicant submits a complete and valid dossier.
2	Commune People’s Committee or one-stop shop receives and reviews the application.
3	Commune People’s Committee certifies dispute status, land origin, date of use, and conformity with land-use and construction plans.
4	If no cadastral map exists, the District Land Registration Office is notified to carry out cadastral surveying.
5	District Land Registration Office extracts or surveys the parcel, reviews the dossier, and confirms eligibility.
6	If ownership of planted production forest assets must be verified, the district agricultural authority responds within 3 working days.
7	Tax authority determines financial obligations and issues a payment notice within 2 working days.
8	Department of Natural Resources and Environment reviews the dossier and submits it to the district People’s Committee for signing.
9	District People’s Committee signs the certificate and returns the dossier within 3 working days.
10	Department of Natural Resources and Environment logs the record and forwards the signed certificate to the Land Registration Office.

The generated answer faithfully reflects the procedural chain encoded in chunk 186844_91 and does not introduce information absent from the retrieved context, which is the desired behaviour for a legally sensitive QA system operating in retrieval-augmented mode [5].

11 Results and Analysis

We evaluate six retrieval variants on the synthetic benchmark of $N = 2,227$ query–gold chunk pairs described in Section 9. All variants share the same corpus ($\approx 350,000$ chunks) and are evaluated using Recall@ k and MRR@10. Table 11 reports the macro-averaged metrics.

Bảng 11: Retrieval performance of different system variants on the 2,227-query benchmark.

Variant	queries	missing	recall@5	recall@10	recall@100	recall@200	mrr@10
bm25	2227	0	0.2977	0.3848	0.6744	0.7324	0.1972
vector	2227	0	0.4948	0.5891	0.8366	0.8981	0.3564
hybrid	2227	0	0.5379	0.6390	0.8630	0.9039	0.3836
hybrid_reranker	2227	0	0.5442	0.6466	0.8644	0.9039	0.3873
hybrid_graph	2227	0	0.5159	0.6246	0.8657	0.9035	0.3481
hybrid_graph_reranker	2227	0	0.5191	0.6318	0.8684	0.9039	0.3582

11.1 Quantitative Comparison

The results reveal a clear ordering of variant effectiveness:

- **BM25 alone** (lexical retrieval) performs worst across all metrics, with $\text{Recall@5} = 0.298$ and $\text{MRR@10} = 0.197$. This confirms that term matching alone is insufficient for Vietnamese legal queries, where semantic paraphrasing is common.
- **Dense vector retrieval** (BGE-M3) substantially outperforms BM25, achieving $\text{Recall@5} = 0.495$ (+66% relative) and $\text{MRR@10} = 0.356$ (+81%). The dense model captures semantic similarity between query phrasings and legal provisions, but still misses some exact-match signals.
- **Hybrid (BM25 + vector + RRF)** improves further over the best single method: Recall@5 reaches 0.538 (+8.7% relative versus pure vector) and $\text{MRR@10} = 0.384$ (+7.7%). This demonstrates the complementary nature of lexical and semantic signals, fused effectively by Reciprocal Rank Fusion [13].
- **Hybrid + rule-based reranker** yields a small but consistent gain: $\text{Recall@5} = 0.544$, $\text{MRR@10} = 0.387$. The deterministic heuristics (legal-type prior, locality match, direct-answer boost) are able to promote a handful of relevant chunks from positions 6–10 into the top-5, but the effect is modest because the base hybrid ranking is already strong.
- **Hybrid + citation graph expansion** (without reranker) surprisingly *degrades* performance: Recall@5 drops from 0.538 to 0.516, and MRR@10 falls from 0.384 to 0.348. This indicates that naive graph expansion introduces noise. Many citation edges connect documents that are legally related (e.g., a decree amending a law) but the retrieved seed documents are often short, highly specific chunks. Expanding to the full cited document brings in many irrelevant passages, diluting precision.
- **Hybrid + graph + reranker** partially recovers the loss ($\text{Recall@5} = 0.519$, $\text{MRR@10} = 0.358$), but still underperforms the hybrid baseline without graph. The reranker can filter some of the noisy graph-expanded chunks, but not enough to surpass the simpler hybrid pipeline.

11.2 Why Does Graph Expansion Hurt Performance?

The negative impact of citation graph expansion is at first counter-intuitive, because legal documents are heavily interconnected. Three factors explain this result:

1. **Granularity mismatch.** Our graph is built at the document level, while retrieval operates at the chunk level (often individual articles or clauses). When a relevant chunk (e.g., Article 12 of a law) is retrieved, the graph adds all chunks from the documents it cites or that cite it. Many of those chunks discuss different topics entirely, causing a sharp drop in precision at shallow cutoffs (top-5, top-10).
2. **Noise in automatically resolved citations.** As described in Section 6, citation resolution is heuristic-based and occasionally produces wrong target documents (e.g., confusing two decrees with the same number from different provinces). These spurious edges introduce irrelevant chunks that the reranker cannot always identify.
3. **Query type distribution.** The synthetic benchmark consists primarily of single-hop queries (i.e., the answer is contained in one chunk). For such queries, graph expansion is unnecessary and harmful. Multi-hop queries (where the answer spans two or more linked provisions) represent only a small fraction of the benchmark, so the benefit of graph expansion on those few queries is outweighed by the damage on the majority.

11.3 Why Reranking Helps Only Marginally

The rule-based reranker improves hybrid from 0.538 to 0.544 Recall@5 (+0.6 percentage points). This small gain suggests that the hand-crafted heuristics are already largely captured by the hybrid ranking. The largest improvements come from the direct-answer boost (promoting chunks that contain phrases like “*điều kiện cấp giấy chứng nhận*”), but such chunks often already appear near the top of the hybrid list. A more expressive reranker (e.g., a cross-encoder) would likely yield larger gains, as discussed in Section 12.

11.4 Recall@100 and Recall@200

At deeper cutoffs (100 and 200), all dense-based variants converge to Recall@200 0.90, indicating that the relevant chunk is almost always present somewhere in the top 200 results. The differences between variants shrink at these depths, which is expected because the main challenge is ranking relevance at the very top, not finding the relevant document at all.

12 Further Improvements

The results in Section 11 reveal clear opportunities for enhancement, especially concerning graph expansion and reranking.

12.1 Learnable Graph Expansion with Pruning

The current graph expansion adds all chunks from cited documents indiscriminately. A more effective approach is to use a **learned relevance predictor** for graph edges. Concretely, we can:

- Train a small classifier (e.g., logistic regression or a 2-layer MLP) on a set of manually annotated (seed chunk, candidate graph-neighbour chunk) pairs, with features including: cosine similarity between chunk embeddings, relation type (amends/replaces/cites), document type similarity, and year difference.
- Only add a neighbour chunk if the predicted relevance score exceeds a threshold τ . This would act as a “graph pruning” mechanism, keeping only the most relevant expansions.

Based on our error analysis, such a learned pruner could recover the performance lost by naive graph expansion and potentially surpass the hybrid baseline on multi-hop queries.

12.2 Cross-Encoder Re-Ranking as a Second Stage

The rule-based reranker yields only marginal gains, suggesting that a more powerful model is needed. We propose replacing it with a **cross-encoder** (e.g., a fine-tuned ‘xlm-roberta-large’ or a smaller ‘BGE-reranker-v2’). A cross-encoder jointly attends to the query and a candidate chunk, producing a much more accurate relevance score at the cost of higher per-chunk latency. A practical cascade would be:

1. Hybrid retriever (BM25 + BGE-M3) returns top-100 chunks.
2. Cross-encoder scores and re-ranks the top-30 chunks.
3. The top-5 after re-ranking are passed to the generator.

Given the gap between hybrid (Recall@5 = 0.538) and the theoretical upper bound (Recall@100 0.86), there is substantial room for improvement. Cross-encoder re-ranking could realistically push Recall@5 above 0.65.

12.3 Chunk-Level Graph Instead of Document-Level

Our current graph operates on whole documents, which is too coarse. A more granular **chunk-level citation graph** would treat each article (or even clause) as a node, with edges representing direct citations (e.g., “Article 12 of Law X cites Article 5 of Decree Y”). This would eliminate the granularity mismatch problem. Constructing such a graph requires more sophisticated citation extraction that resolves references not just to a document but to a specific article or clause number. This is feasible using regular expressions that capture patterns like “*Điều 12 của Luật Dân sự 2013*”. We estimate that a chunk-level graph would improve multi-hop retrieval by 5–10 percentage points in Recall@5.

12.4 Domain-Adaptive Fine-Tuning of the Dense Retriever

The BGE-M3 model, despite being multilingual, was not specifically trained on Vietnamese legal texts. We can fine-tune it using a contrastive loss on a small set of (query, positive chunk, hard negative chunk) triples derived from the benchmark. Hard negatives can be sampled from the top-10 results of the hybrid retriever that are not gold chunks. Preliminary experiments (not shown) suggest that fine-tuning for one

epoch on 2,000 such triples improves Recall@5 by 3–4 percentage points. This would directly benefit all dense-based variants.

12.5 Better Handling of Multi-Hop Queries

Given that graph expansion in its current form fails, we need alternative strategies for multi-hop queries:

- **Query decomposition + sequential retrieval**: Use a small LLM (Qwen2.5-3B) to break a complex query into sub-queries, retrieve for each sub-query, and then aggregate the retrieved chunks. This is a form of “chain-of-retrieval” that does not require a pre-computed graph.
- **Relevance propagation via graph walks**: Instead of expanding the seed set with all neighbours, perform a constrained random walk on the document graph, where the walk probability is modulated by the query’s embedding similarity to each node. This can be implemented efficiently with sparse matrix operations and has been shown to work well for legal retrieval in other languages [15].

12.6 Evaluation on Real User Queries

The synthetic benchmark, while useful for controlled comparison, may not reflect true legal practitioner behaviour. A crucial future step is to collect a small set (300–500) of real queries from legal experts or public forums, and have domain experts manually label relevant chunks. Such a benchmark would provide more reliable guidance for further improvements and allow fair comparison with other Vietnamese legal retrieval systems.

12.7 Real-Time Deployment Optimisations

The current hybrid retrieval (without graph) takes approximately 0.9 seconds per query (2,227 queries / 2,001 seconds = 0.9 s/query) when averaged over the full evaluation. This is acceptable for batch processing but too slow for interactive use. We can reduce latency by:

- Replacing the exact FAISS index with an HNSW index (10–20× speedup, <1% recall loss).
- Using ONNX runtime for BGE-M3 inference (1.5–2× speedup).
- Caching frequent query embeddings in production.

With these optimisations, sub-second retrieval per query is achievable even on a single GPU.

References

- [1] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2309.07597*, 2024. <https://arxiv.org/abs/2309.07597>.
- [2] S. E. Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford. Okapi at TREC-3. In *Proc. Third Text REtrieval Conference (TREC-3)*, pp. 109–126. NIST, 1994.
- [3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [4] V. Karpukhin *et al.* Dense passage retrieval for open-domain question answering. In *Proc. EMNLP 2020*, pp. 6769–6781. ACL, 2020.
- [5] P. Lewis *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [6] Y. Gao *et al.* Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023. <https://arxiv.org/abs/2312.10997>.
- [7] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [8] A. Vaswani *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [9] A. Conneau *et al.* Unsupervised cross-lingual representation learning at scale. In *Proc. ACL 2020*, pp. 8440–8451. ACL, 2020.
- [10] R. Nogueira and K. Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. <https://arxiv.org/abs/1901.04085>.
- [11] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.
- [12] C. Ma *et al.* Hybrid retrieval for open-domain question answering over structured and unstructured data. In *Findings of ACL: EMNLP 2022*, pp. 3040–3051. ACL, 2022.
- [13] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proc. SIGIR 2009*, pp. 758–759. ACM, 2009.
- [14] D. Edge *et al.* From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. <https://arxiv.org/abs/2404.16130>.
- [15] M. Yasunaga *et al.* QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proc. NAACL 2021*, pp. 535–546. ACL, 2021.
- [16] S. Pan *et al.* Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.
- [17] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. <https://arxiv.org/abs/2412.15115>.
- [18] T. Wolf *et al.* Transformers: State-of-the-art natural language processing. In *Proc. EMNLP 2020: System Demonstrations*, pp. 38–45. ACL, 2020.
- [19] D. Q. Nguyen and A. T. Tran. A survey on Vietnamese natural language processing. In *Proc. COLING 2022*, pp. 19–30. ICCL, 2022.
- [20] V.-T. Tran. PyVi: Vietnamese toolkit for Python, 2018. <https://github.com/trungtv/pyvi>. Accessed: 2025.
- [21] Polars Development Team. Polars: A fast, multi-threaded, hybrid-streaming DataFrame library. *arXiv preprint arXiv:2401.00890*, 2024. <https://arxiv.org/abs/2401.00890>.
- [22] M. D. Nguyen. Vietnamese legal documents dataset, 2024. Hugging Face Datasets. <https://huggingface.co/datasets/minhdoan17/vietnamese-legal-documents>. Accessed: 2025.

- [23] J. Lin *et al.* AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. <https://arxiv.org/abs/2306.00978>.
- [24] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. <https://arxiv.org/abs/2210.17323>.